

Session 06 – Unsupervised learning

Hector J. Hortua, PhD

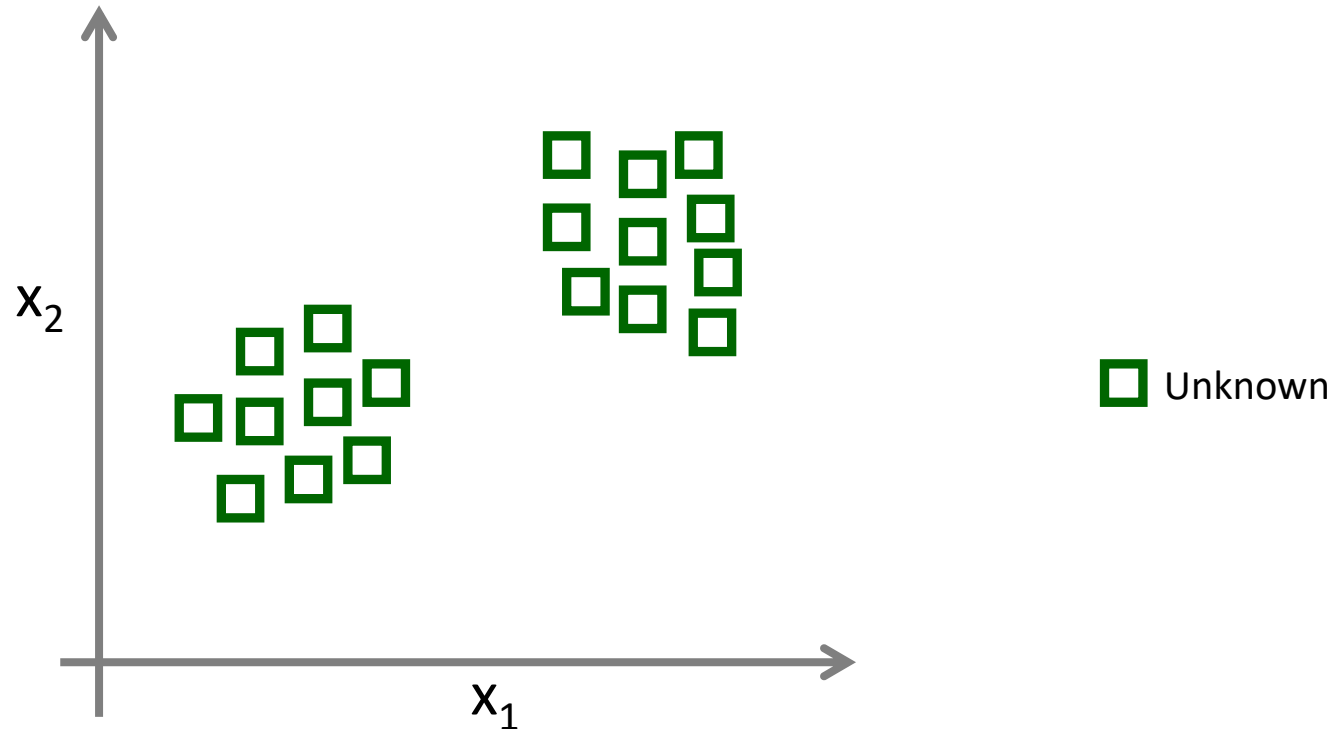
Learning outcomes

By the end of this session we should be able to:

- Understand the kind of problems in which unsupervised learning is best suited.
- Describe several unsupervised learning algorithms, how they are optimised and some application examples.

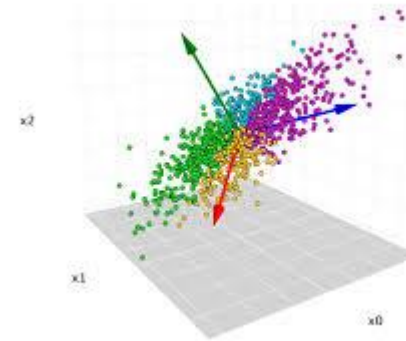
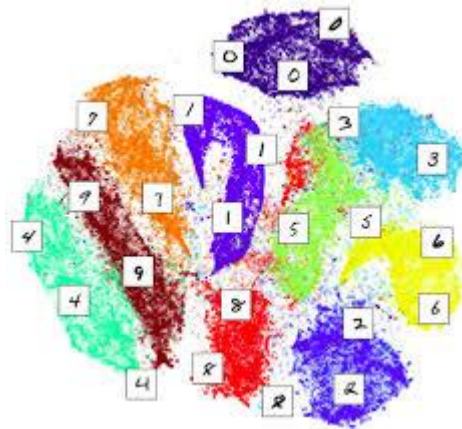
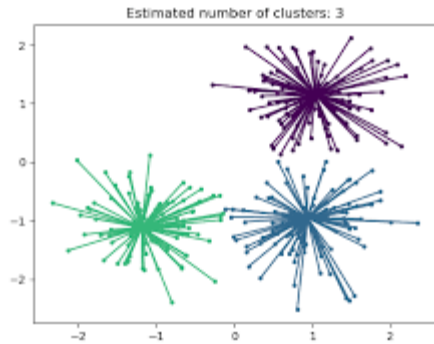
Unsupervised learning

- **No labels** are given to the learning algorithm to find structure in its input
- The goal is to discovering the **hidden** patterns in the data



Unsupervised learning tasks

- Clustering – e.g. k-means, Gaussian Mixture Models (GMM),..
- Data visualisation – e.g. principal component analysis (PCA), t-SNE, UMAP,..
- Dimensionality reduction – e.g. PCA, autoencoders, embeddings
- Novelty detection – e.g. Anomaly detection

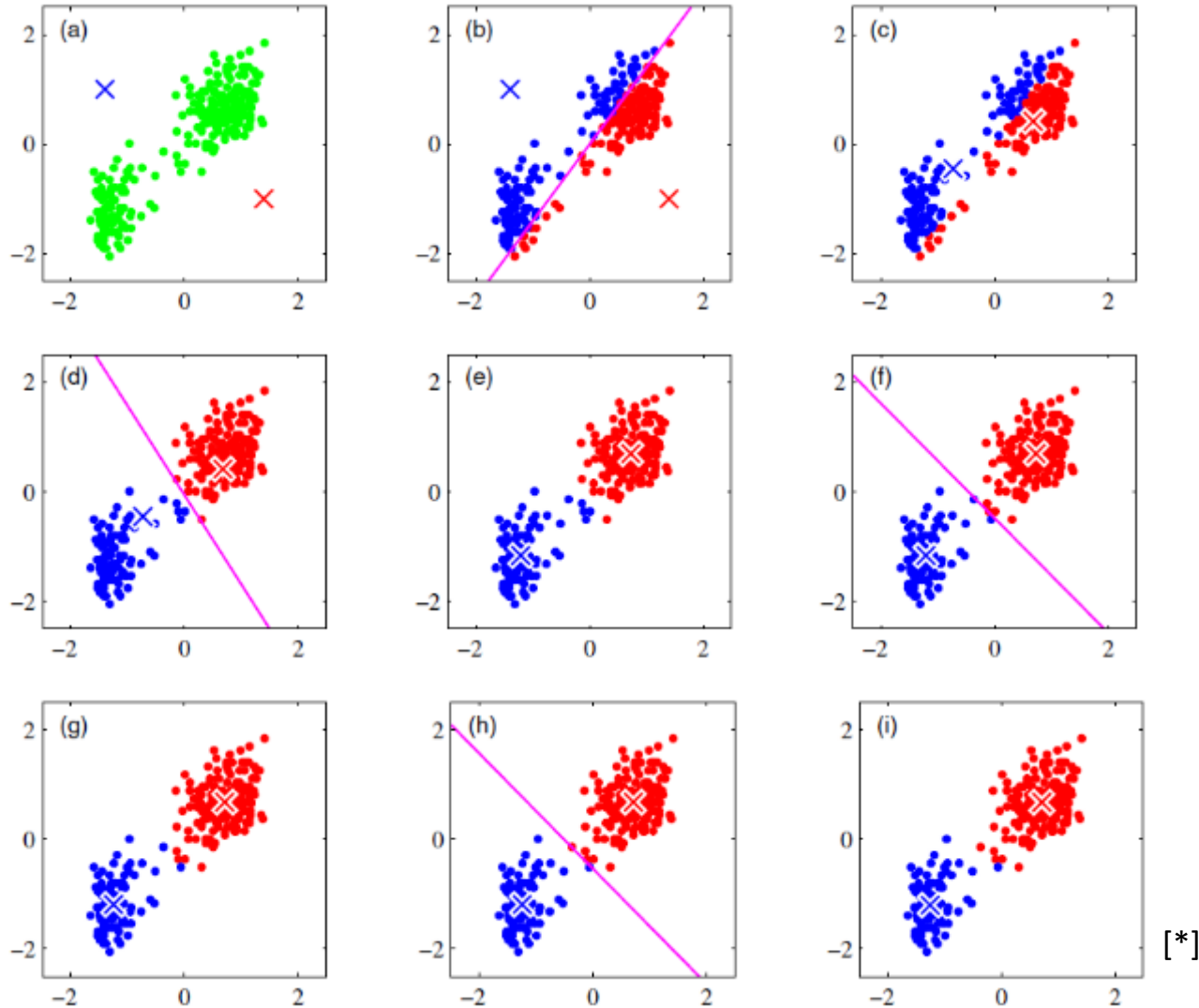


What is Clustering?

- Clustering is alternatively called ‘grouping’
- It attaches a label to each observation or data point in a set
- The idea behind clustering is to assign the same label to data points that are “close” to each other
 - In a cluster, inter-point distances are small compared with the distances to points outside of the cluster ^[*]
- Thus, clustering algorithms rely on a distance metric between data points
- Sometimes it is said that for clustering, the distance metric is more important than the actual clustering algorithm

[*] Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. [page 424]

Illustration of K -means with 2 clusters



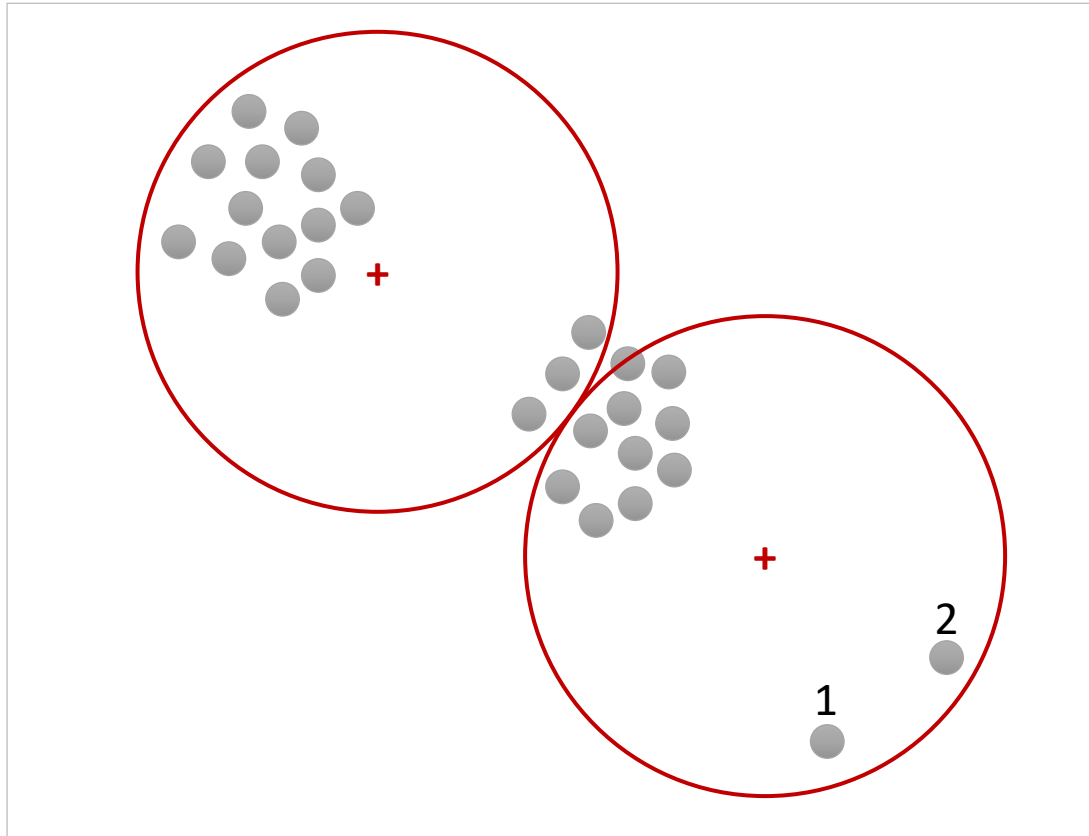
(a) Green points: dataset. Red and blue crosses: initial centres μ_1 and μ_2 .

(b) Each data point is assigned either to the red or to the blue cluster, according to which cluster centre is nearer.

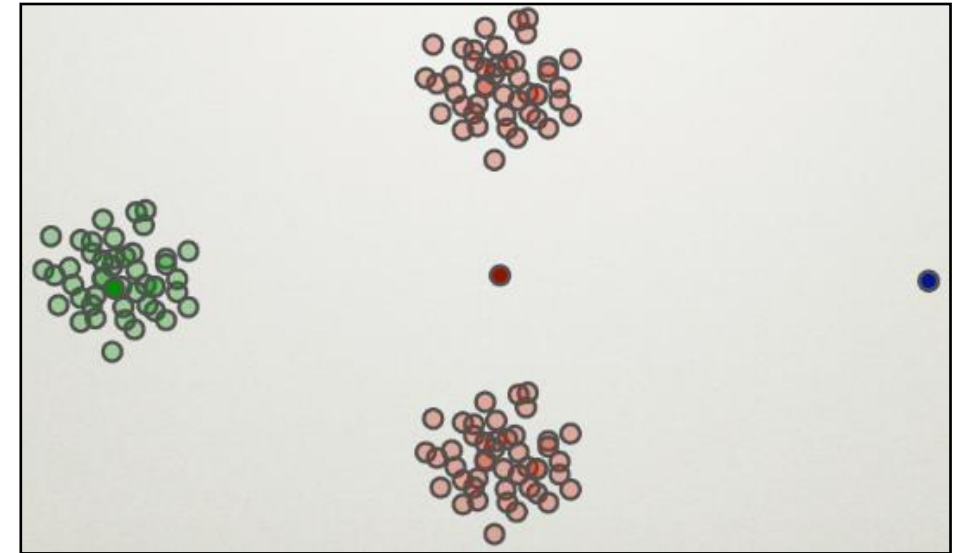
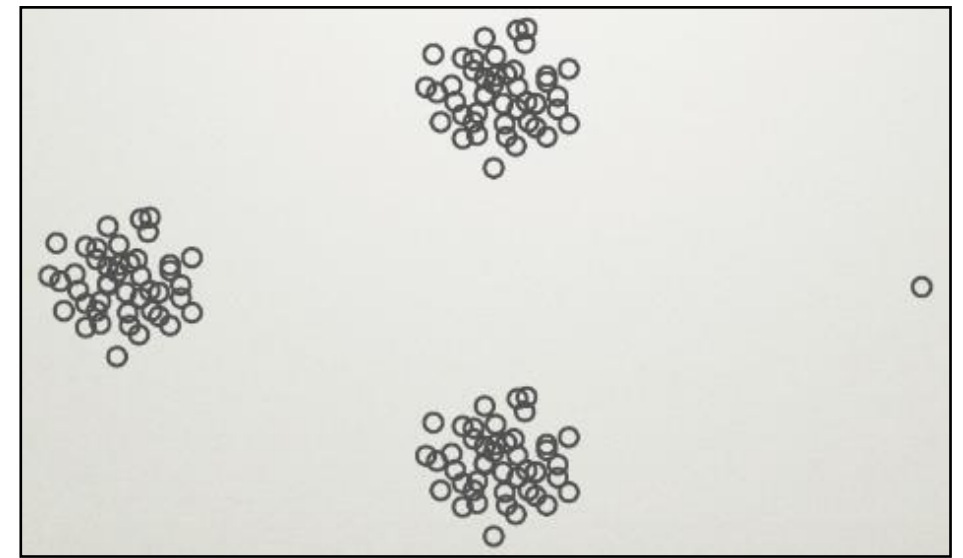
(c) Each cluster centre is re-computed to be the mean of the points assigned to the corresponding cluster.

(d)– (i) show successive steps through to final convergence of the algorithm.

K -means in presence of outliers



K-means is extremely sensitive to outliers.

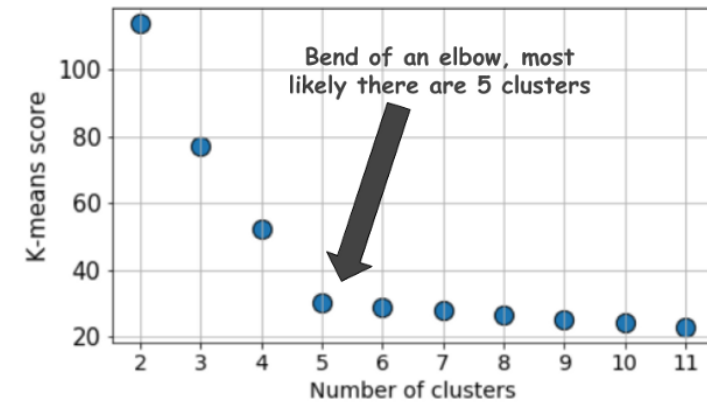


e.g. 3 clusters

Choosing the number of clusters

- The elbow method – the idea of the elbow method is to run k-means clustering on the dataset for a range of values of k (say, k from 1 to 10), and for each value of k calculate the cost function J (inertia).
- Then, plot a line chart of the J for each value of k . If the line chart looks like an arm, then the "elbow" on the arm is the value of k that is the best.
- The idea is that we want a small J , but not 0, so the number of clusters does not correspond to the number of data points.

The elbow method for determining number of clusters



For a set of k clusters, the cost function is:

$$J = \sum_{j=1}^k \sum_{i=1}^{n_j} \|\mathbf{x}_i^{(j)} - \mathbf{c}_j\|^2$$

Where:

- k = number of clusters
- n_j = number of points in cluster j
- $\mathbf{x}_i^{(j)}$ = the i -th data point in cluster j
- $\mathbf{c}_j$ = centroid (mean) of cluster j
- $\|\cdot\|^2$ = squared Euclidean distance

K -means: summary

- Algorithmically, very **simple** to implement
- K -means converges, but it finds a **local minimum**
 - K -means ++ can be used for choosing a good initial set of values that leads to a potentially better solution
- Works only for **numerical observations**
- Works well only for **round shaped**, and of **roughly equal sizes/density** clusters
- K is a **user input**; but there are approaches that can be used to estimate it.
- Outliers can considerable trouble K -means

Metric	Range	Best Value	Formula
Inertia (WCSS)	$[0, \infty)$	Lower	$\sum \ x_i - c_j\ ^2$
Silhouette Score	$[-1, 1]$	Higher (+1)	$(b - a) / \max(a, b)$
Davies-Bouldin Index	$[0, \infty)$	Lower (0)	$(1/k) \sum \max((\sigma_i + \sigma_j) / d(c_i, c_j))$

Gaussian Mixture Models

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N z_{nk} x_n$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N z_{nk} (x_n - \mu_k)(x_n - \mu_k)^T$$

$$\pi_k = \frac{N_k}{N}$$

$$N_k = \sum_{n=1}^N z_{nk}$$

- We will estimate $p(x)$ as a weighted sum of Gaussians
- Formally:
$$P(x) = \sum_{k=1}^K \pi_k \cdot N_{\mu_k, \Sigma_k}(x)$$
 - When N denotes the normal (or Gaussian) distribution
 - The π_k weights are non-negative and add up to 1
 - The number K of the Gaussians is fixed (but it could be optimized as well)
- By increasing the number of components the curve defined by the mixture model can take basically any shape, so it is much more flexible than just one Gaussian

Calculating the responsibility values

- We would need a soft clustering algorithm
- But from *given* Gaussian curves we could also calculate the z_{nk} weights
- The z_{nk} "responsibility" values express how much a point x_n belongs to the k -th Gaussian
- We can formalize this as

$$z_{nk} = \frac{\pi_k N(x|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x|\mu_j, \Sigma_j)}$$

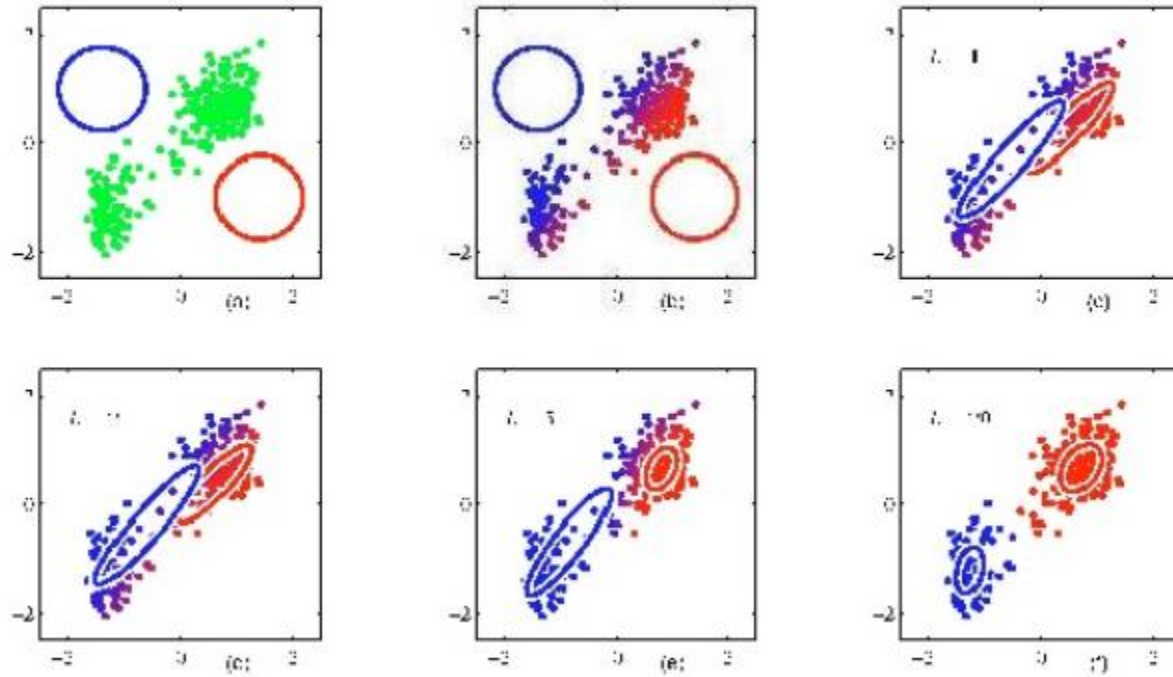
- This is simply the likelihood of the k -th Gaussian
 - Which is divided by the likelihood of the full model just to normalize it to the $[0,1]$ range
- So the Gaussian components themselves define a soft clustering
 - But for this we should know the Gaussians, that is, the μ_k és Σ_k values!

The iterative EM training process

- What do we have now?
 - We can easily fit the Gaussians (estimate μ_k and Σ_k) if the responsibilities are given
 - We can easily calculate the responsibilities if the Gaussians (μ_k and Σ_k) are given
- We can turn it into an iterative process – the EM process
 - 0. step: Initialization of μ_k and Σ_k (randomly, or by K-means)
 - 1. step (E step or Expectation step): estimate the responsibilities
 - 2. step (M step or Maximization step): maximize the log-likelihood of each Gaussian by re-calculating μ_k and Σ_k
 - Go back to step 1 (μ_k and Σ_k changed, so we must re-calculate the responsibilities!)

Example

with random initialization:

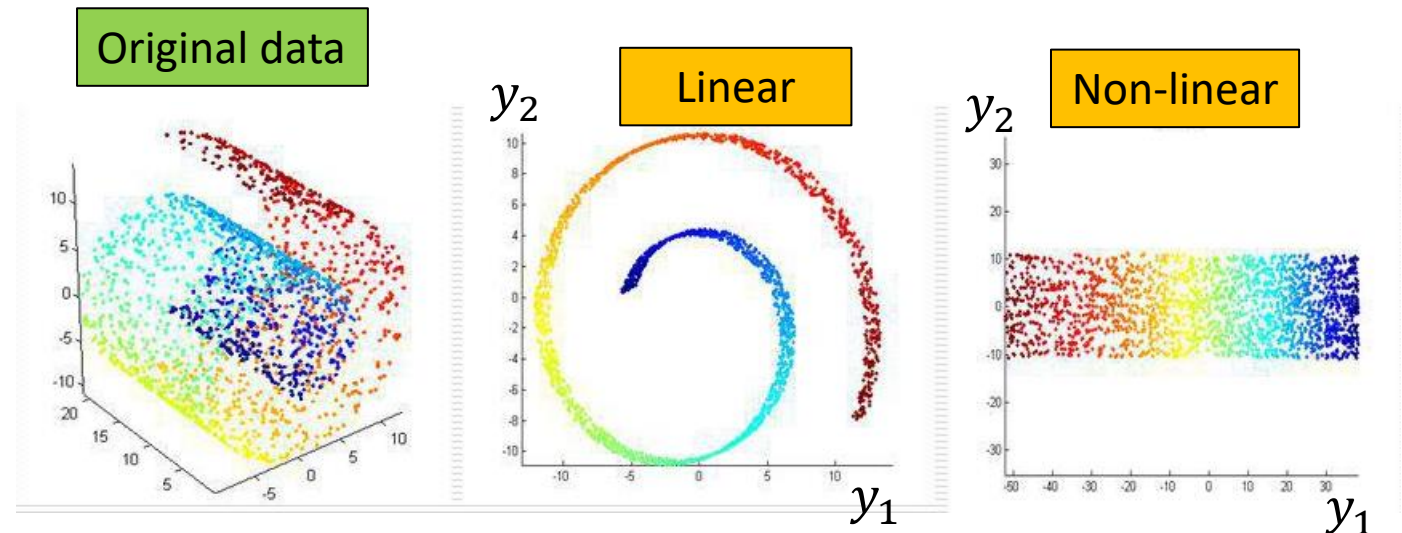
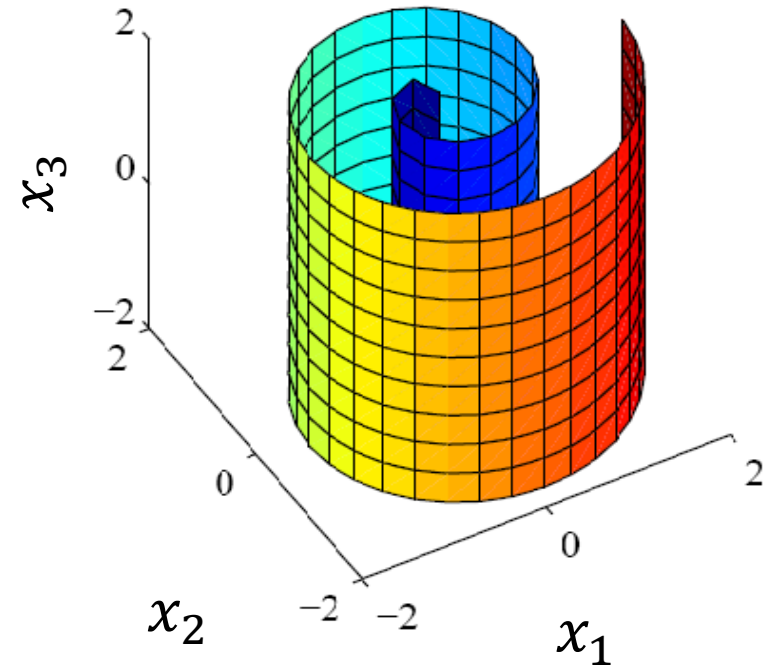


Dimensionality reduction

- Dimensionality reduction algorithms:
 - Map a high dimensional data into a lower dimension while preserving their structure.

$$\mathcal{X} = \{x_1, x_2, \dots, x_n \in \mathbb{R}^h\} \rightarrow \mathcal{Y} = \{y_1, y_2, \dots, y_n \in \mathbb{R}^l\}$$
$$\min_{\mathcal{Y}} C(\mathcal{X}, \mathcal{Y})$$

- They are used for:
 - Data visualisation
 - Performance improvement
 - Curse of dimensionality
- There are many algorithms
 - Principal components analysis
 - T-SNE
 - Autoencoders
 - Etc



Curse of Dimensionality

Partition the input space, put each data point in a cell, then search the cell containing the query q and its neighboring cells.

$$3^2 = 9$$

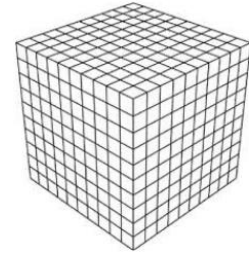
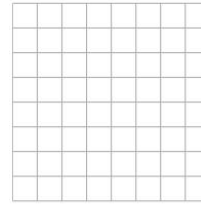
cells to examine in 2D

$$3^3 = 27$$

cells to examine in 3D

$$3^m$$

cells to examine in m D

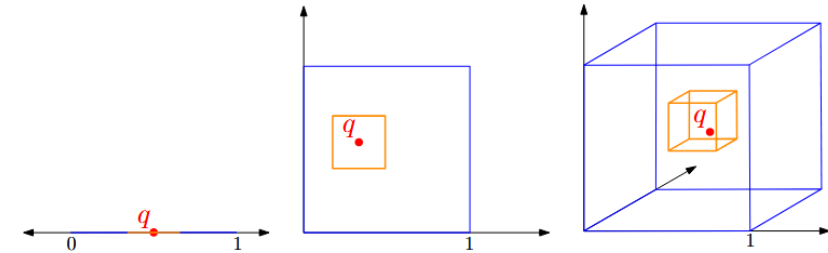


Partitioning helps in low dimension, but the number of surrounding cells grows as 3^m .

with each new dimension, the “nearby” search neighborhood multiplies—not adds.

To capture the same fraction of points, the search cube gets huge

For n points uniformly in $[0,1]^m$, the expected edge length of a cube containing fraction f of the points is $E_m(f) = f^{1/m}$. As m grows, $f^{1/m}$ moves toward 1.



Same idea in 1D, 2D, and 3D: the search region expands along every dimension.

1D

0.1%

edge length

2D

3.1%

edge length

3D

10%

edge length

4D

17.7%

edge length

10D

50.1%

edge length

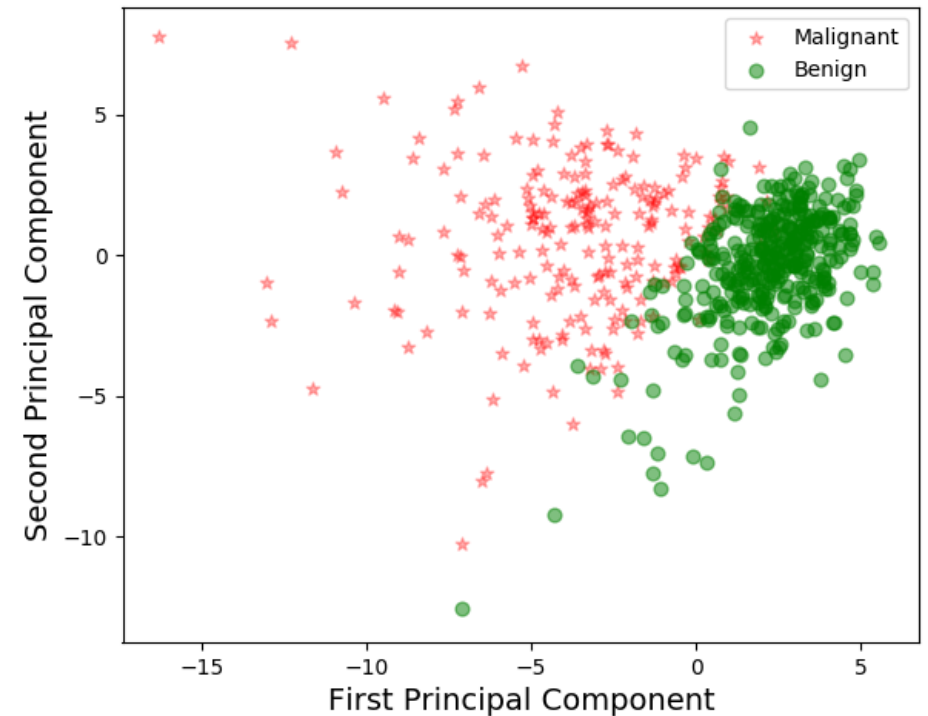
With 5,000 random points, to capture just 5 nearest neighbors ($f=5/5000=0.001$), the search radius along each dimension jumps from 0.1% in 1D to about 50% in 10D.

Principal Component Analysis (PCA) – Data visualisation

- Principal components:

$$\mathbf{Z} = \mathbf{X}^T \boldsymbol{\phi} = \boldsymbol{\phi}^T \mathbf{X}$$

- In PCA, the first principal component represents the direction in the data that has the largest variance, and subsequent principal components represent directions with decreasing variance.
- We use the first two components to project the data into a visualisation map: the **score plot**
- The hope is that the two first PCs explain the data variance sufficiently.
- $\boldsymbol{\phi}$ is a matrix of weights. Each row is called a loading vector.



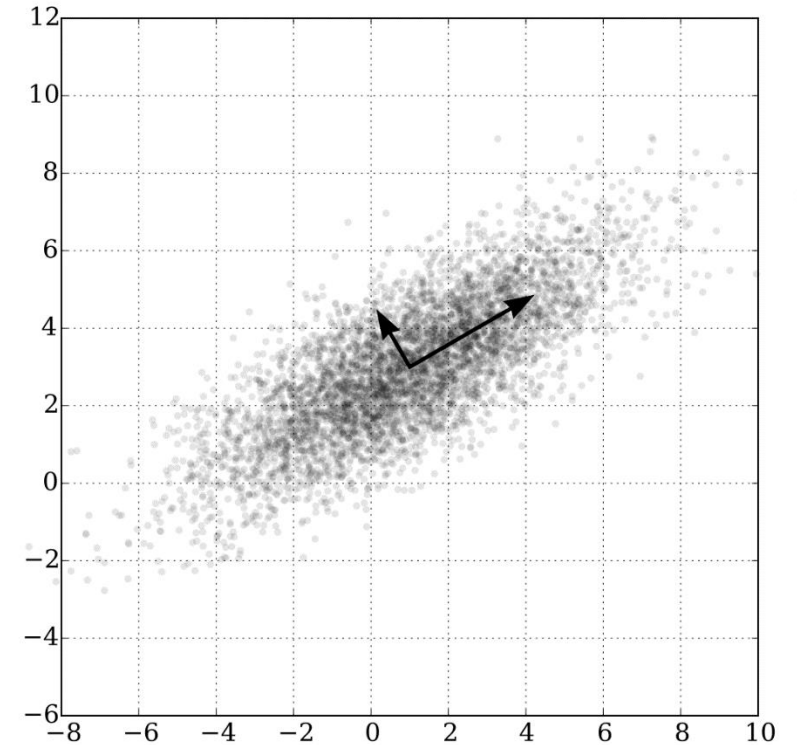
PCA – General formulation

- Assuming a zero-centred data, it is possible to find the eigen decomposition:

$$\lambda \boldsymbol{\phi} = C \boldsymbol{\phi}$$

- Where:
 - $C = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$ is the covariance matrix of the data.
 - λ is a vector of eigenvalues.
 - $\boldsymbol{\phi}$ is a matrix of eigen column vectors.
- By ordering the λ s, we can obtain new rotated data components such that they are ordered in terms of how much variance is explained.

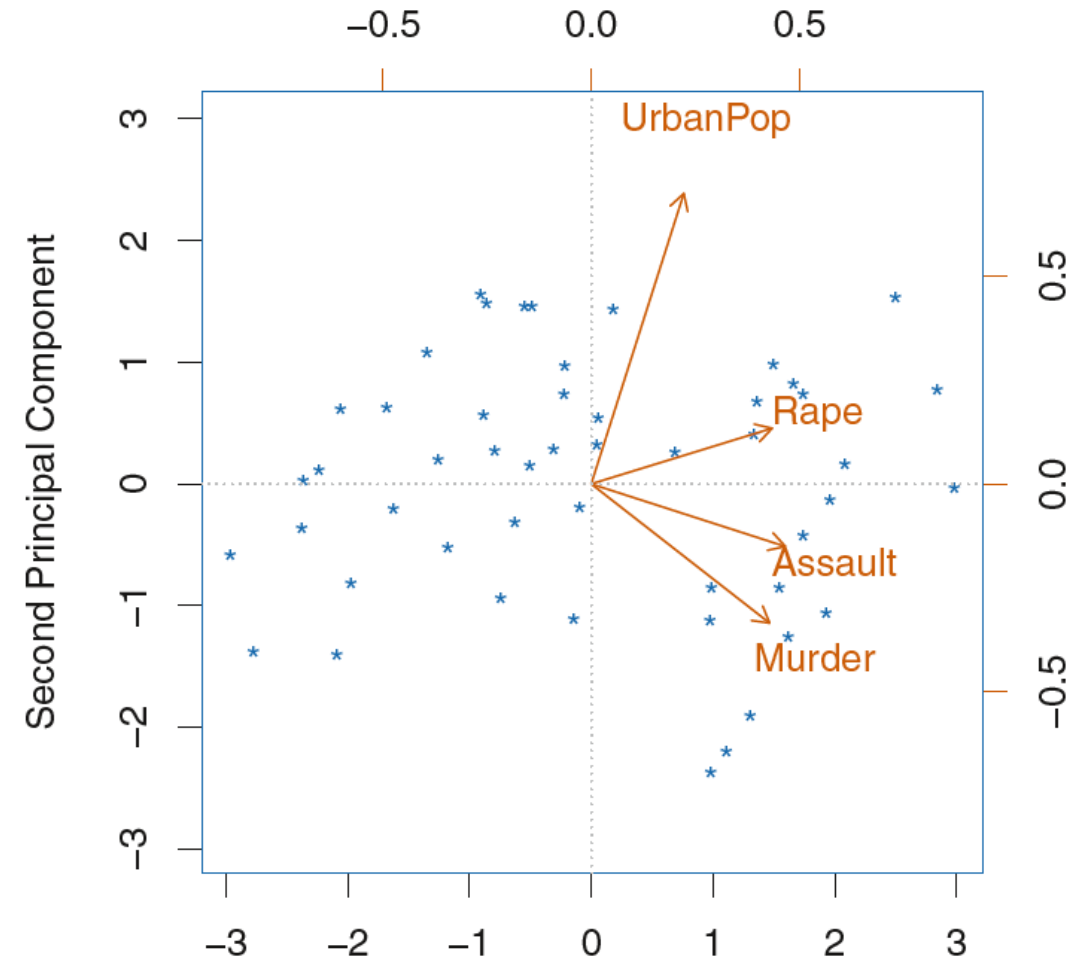
$$\lambda_1 > \lambda_2 > \dots \lambda_d$$



PCA – Data visualisation

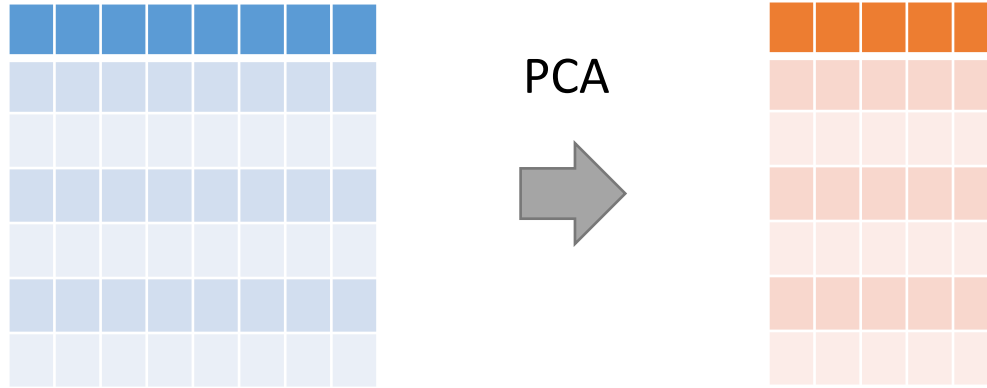
- Loading vectors (ϕ) can also be plotted.
- **PCA biplots** – Combination of score and loading plots.
- Interpretation:
 - The magnitude of a loading vector indicates how much the corresponding variable influences the PCs (e.g. “UrbanPop” influences more PC2 than PC1)
 - Two variables are positively correlated if the angle between their loading vectors is small (e.g. “Assault” and “Murder”).
 - If the loading vectors are perpendicular, then the variables are uncorrelated (e.g. “UrbanPop” and “Assault”).
 - But if the angle is close to 180 degrees, then the correlation is negative.

	PC1	PC2
Murder	0.5358995	-0.4181809
Assault	0.5831836	-0.1879856
UrbanPop	0.2781909	0.8728062
Rape	0.5434321	0.1673186



PCA – Dimensionality reduction

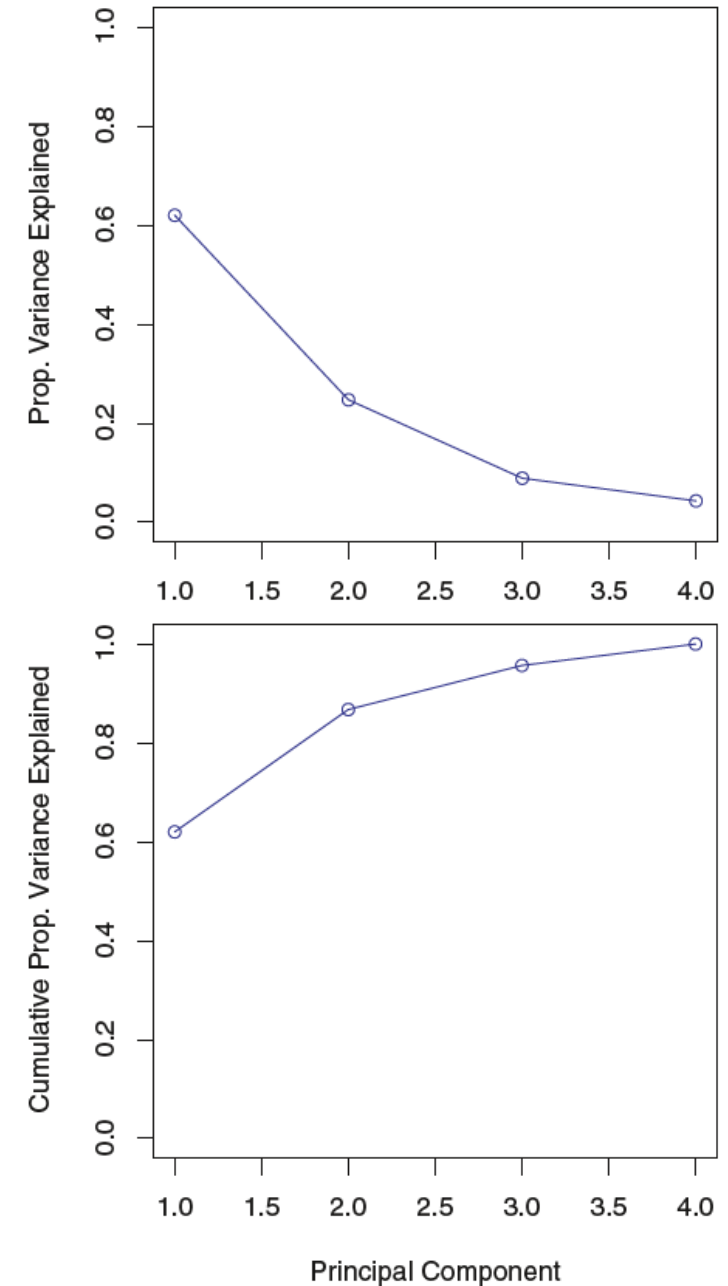
- The number of PCs cannot be higher than the number of dimensions.
- In general, PCA is used for dimensionality reduction (feature extraction)



- It is a plot of the number of components vs the proportion of variance explained (PVE):

$$PVE = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{i=1}^n \sum_{j=1}^p x_{ij}^2}$$

- It is assumed that the optimal number of components should explain at least 80% of the variance.



Steps of PCA

- Let $\bar{X}$ be the mean vector (taking the mean of all rows)

- Adjust the original data by the mean

$$X' = X - \bar{X}$$

- Compute the covariance matrix C of adjusted X
- Find the eigenvectors and eigenvalues of C .

- For matrix C , vectors $\mathbf{e}$ (=column vector) having same direction as $C\mathbf{e}$:
 - *eigenvectors* of C is $\mathbf{e}$ such that $C\mathbf{e} = \lambda\mathbf{e}$,
 - λ is called an *eigenvalue* of C .

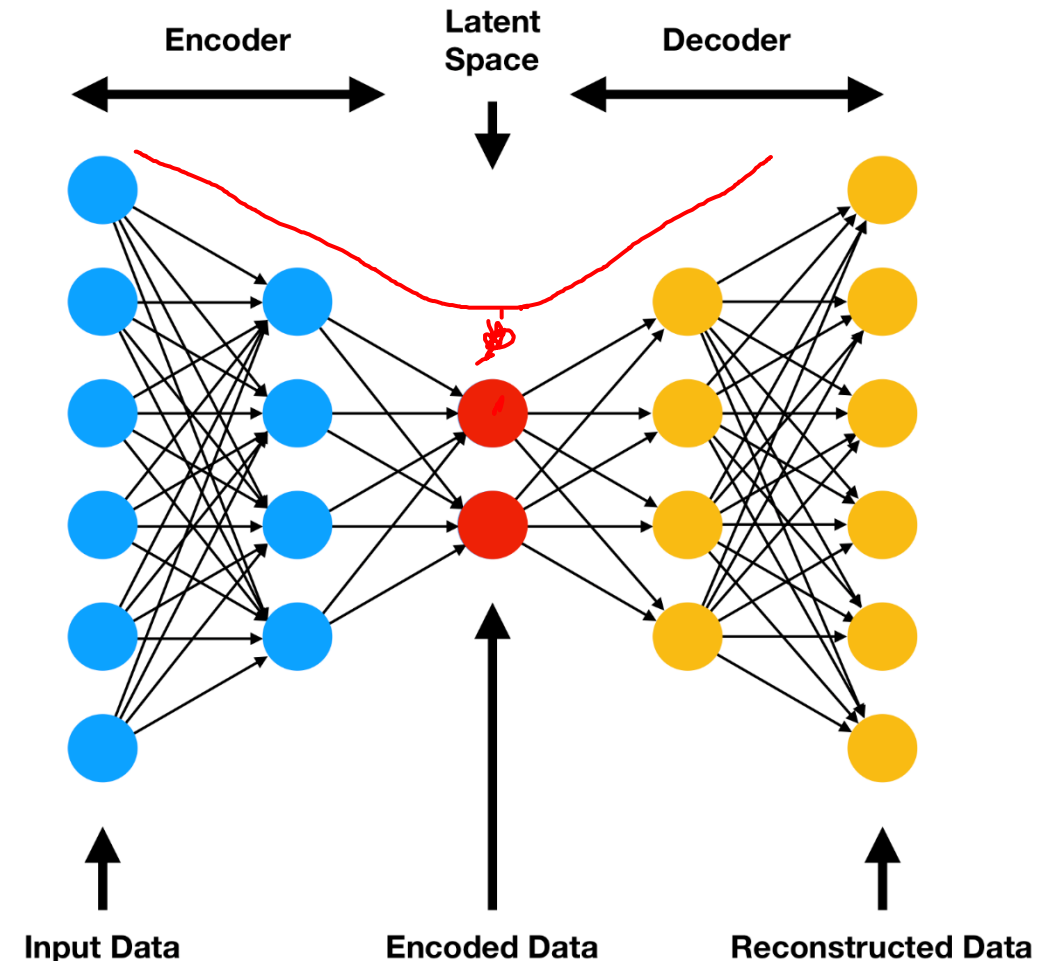
- $C\mathbf{e} = \lambda\mathbf{e} \Leftrightarrow (C - \lambda I)\mathbf{e} = 0$

- **Most data mining packages do this for you.**

- Retrieving old data (e.g. in data compression)
 - $RetrievedRowData = (RowFeatureVector^T \times FinalData) + OriginalMean$
 - Yields original data using the chosen components

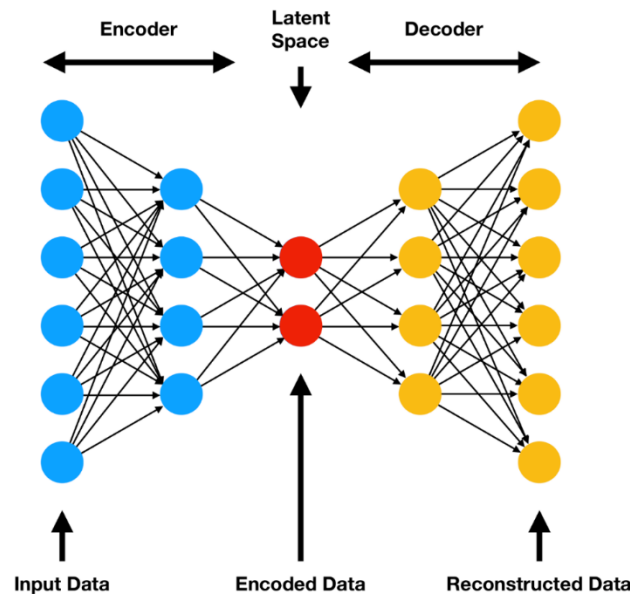
Autoencoders

- Autoencoders are neural networks that can be used to reduce the data into a low dimensional latent space by stacking multiple non-linear transformations(layers).
- They have a encoder-decoder architecture.
- **Encoder** – maps the input to a (usually low-dimensional) latent space.
- **Decoder** – reconstructs the input from the latent space.
- They are trained using back propagation for accurate reconstruction of the input.
- If the latent space has lower dimensions than the input, autoencoders can be used for dimensionality reduction.
- By intuition, these low dimensional latent variables should encode most important features of the input since they are capable of reconstructing it.

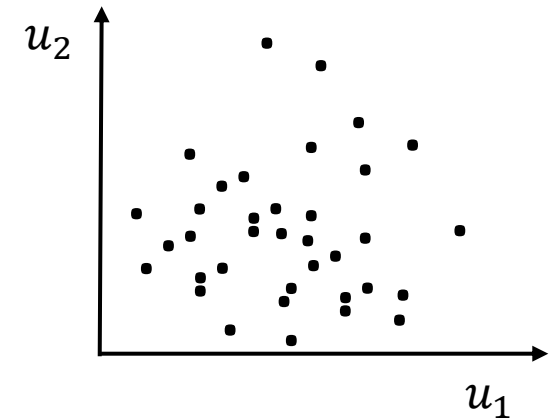
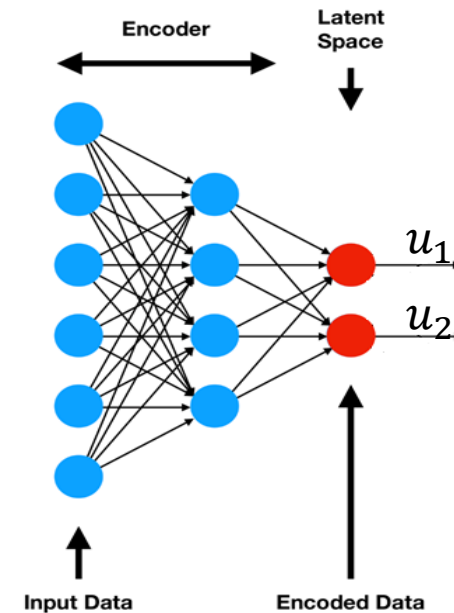


Autoencoders – dimensionality reduction-Anomaly detection

Training mode



Predicting mode

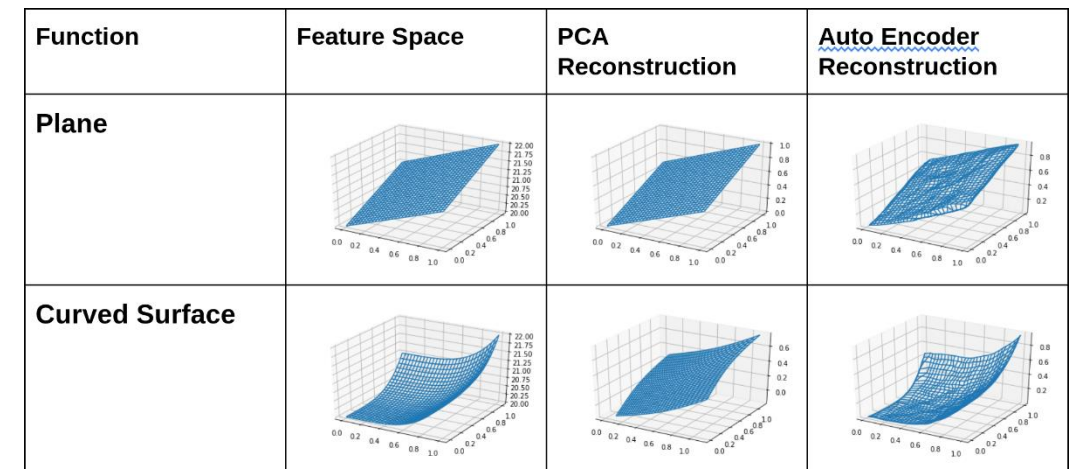
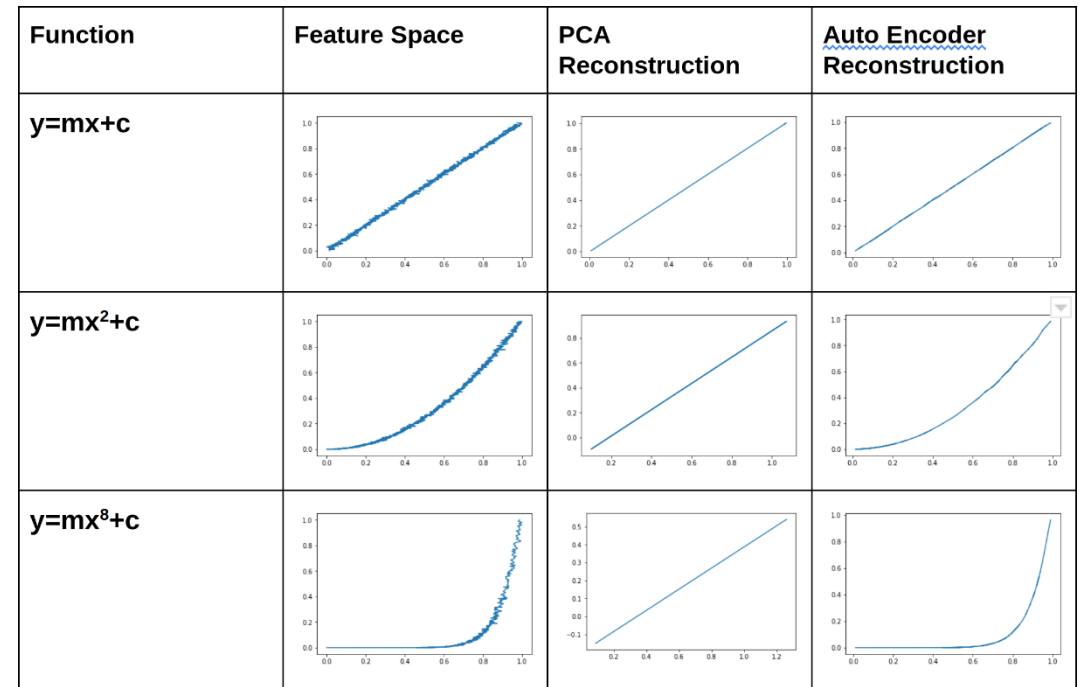


- In training mode, reconstruction error is minimised by error backpropagation optimisation algorithms
- In predicting mode, only the encoder part is used. The output is the encoded data which lay on a low-dimensional space

Low-dimensional data representation. If U is 2-D, the data can be visualised (e.g. producing a scatter plot)

Autoencoders vs PCA

- PCA is essentially a linear transformation.
- Autoencoders are capable of modelling complex non linear functions.
- PCA features are totally linearly uncorrelated with each other since features are projections onto the orthogonal basis.
- Autoencoded features might have correlations since they are just trained for accurate reconstruction.
- PCA is faster and computationally cheaper than autoencoders.
- A single layered autoencoder with a linear activation function is very similar to PCA.
- Autoencoder is prone to overfitting due to high number of parameters. (though regularization and careful design can avoid this)



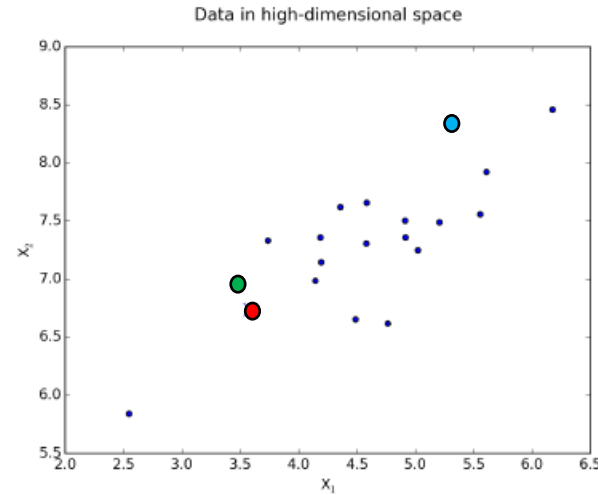
Stochastic neighbour embedding (SNE)

- SNE converts Euclidean distances to similarities, that can be interpreted as probabilities.

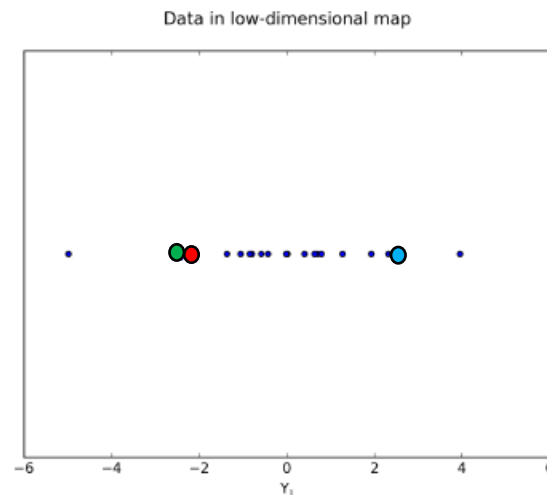
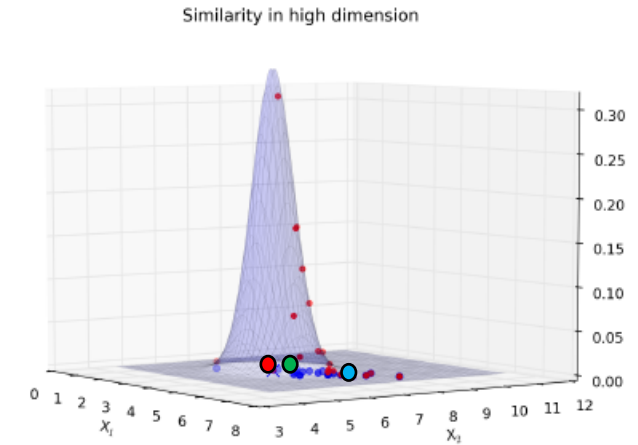
$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

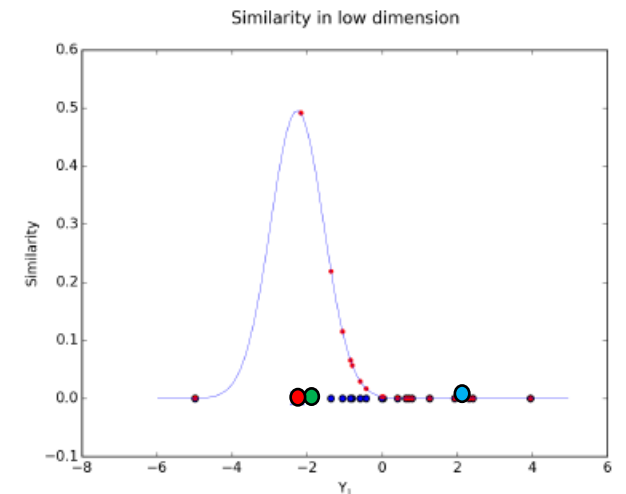
$$p_{i|i} = 0, q_{i|i} = 0$$



$$p_{j|i}$$
$$\Leftrightarrow$$



$$q_{j|i}$$
$$\Leftrightarrow$$



Kullback-Leiber Divergence

- KL divergence measures the faithfulness with which $q_{j|i}$ models $p_{j|i}$.
- Kullback-Leiber Divergence (KL) compares two distributions.

$$\mathcal{C} = \sum_i KL(P_i || Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- KL divergence is asymmetric
- KL divergence is always positive.
- We have our minimization problem: $\min_y \mathcal{C}(\mathcal{X}, \mathcal{Y})$.

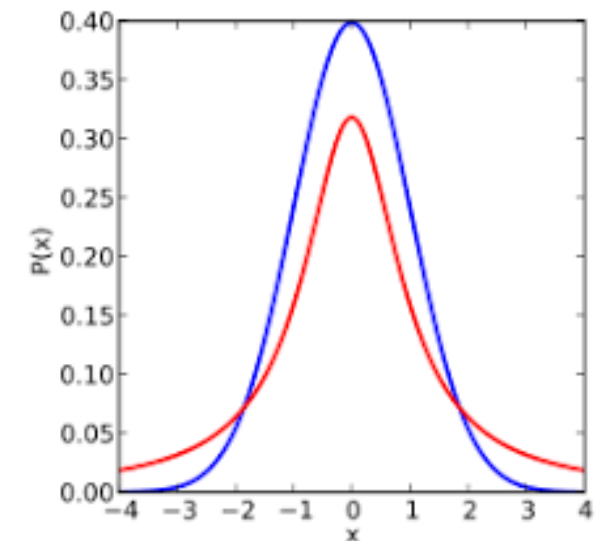
t-Distributed Stochastic Neighbour Embedding (t-SNE)

- It is a technique for dimensionality reduction that is particularly well suited for the visualisation of high-dimensional datasets.
- The t-SNE algorithm comprises two main stages:
 1. t-SNE constructs a probability distribution over pairs of high-dimensional objects in such a way that similar objects are assigned a higher probability while dissimilar points are assigned a lower probability.
 2. t-SNE defines a similar probability distribution over the points in the low-dimensional map, and it minimizes the Kullback–Leibler divergence (KL divergence) between the two distributions with respect to the locations of the points in the map.
 - Student t-distribution has a longer tail than the normal distribution.
 - It is used to allow dissimilar objects to be modelled far apart in the map

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}$$

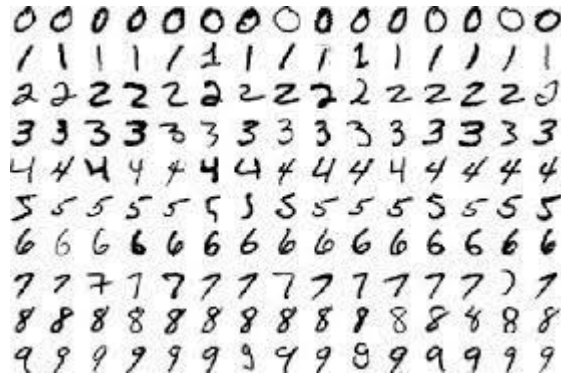
$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

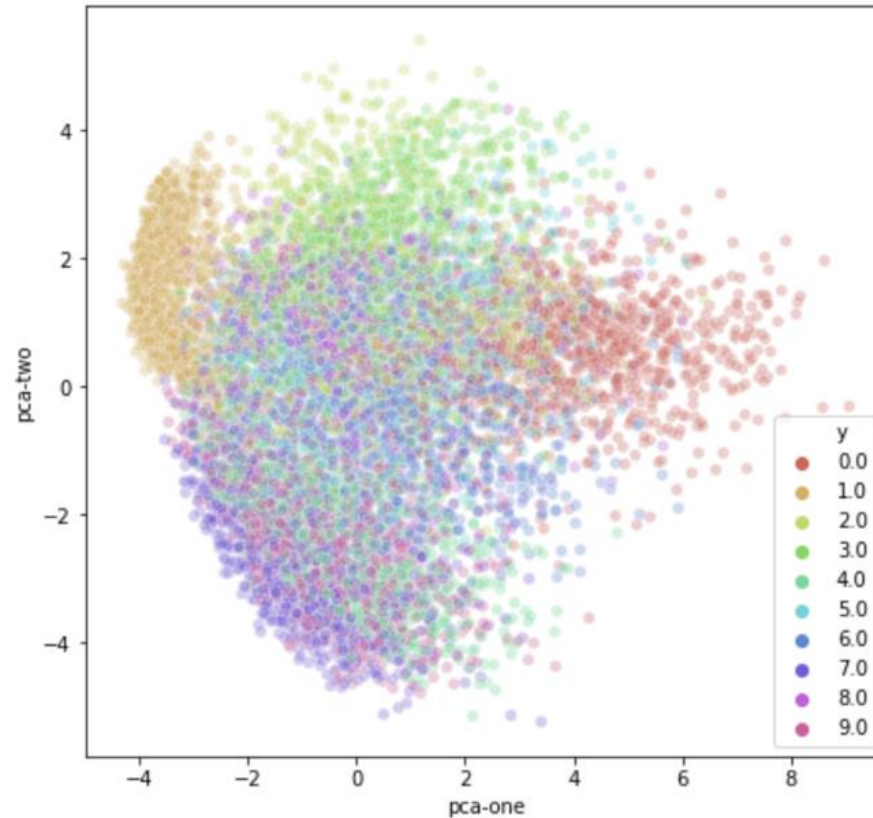


PCA vs t-SNE

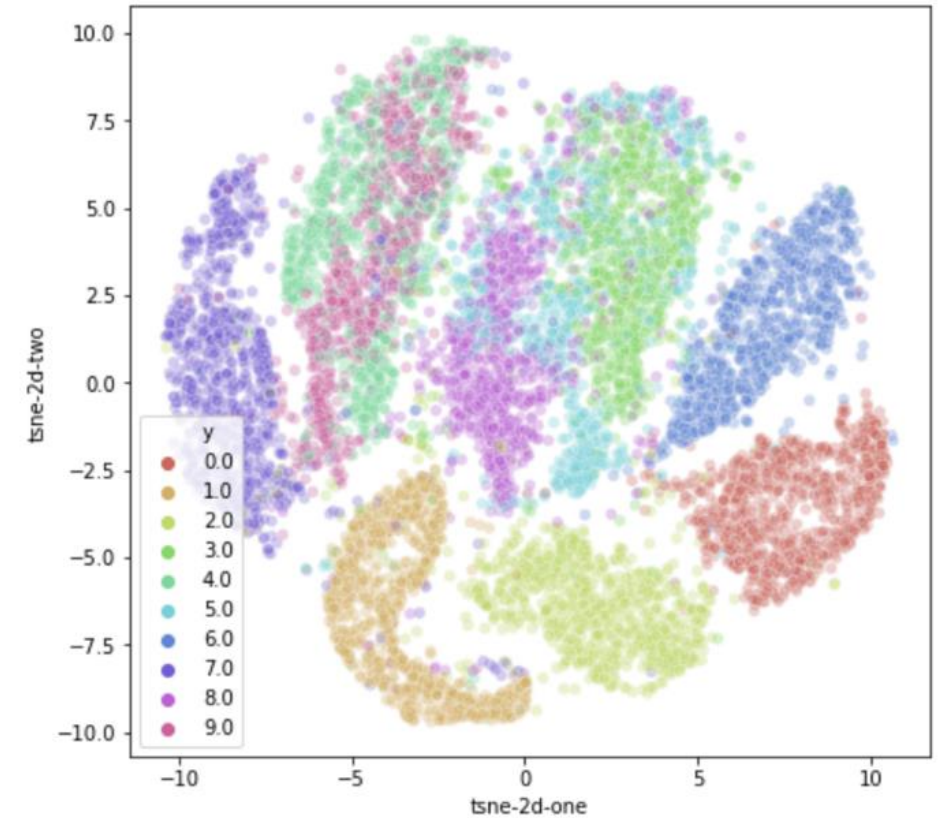
- MNIST data: handwritten digit classification problem.



PCA visualisation

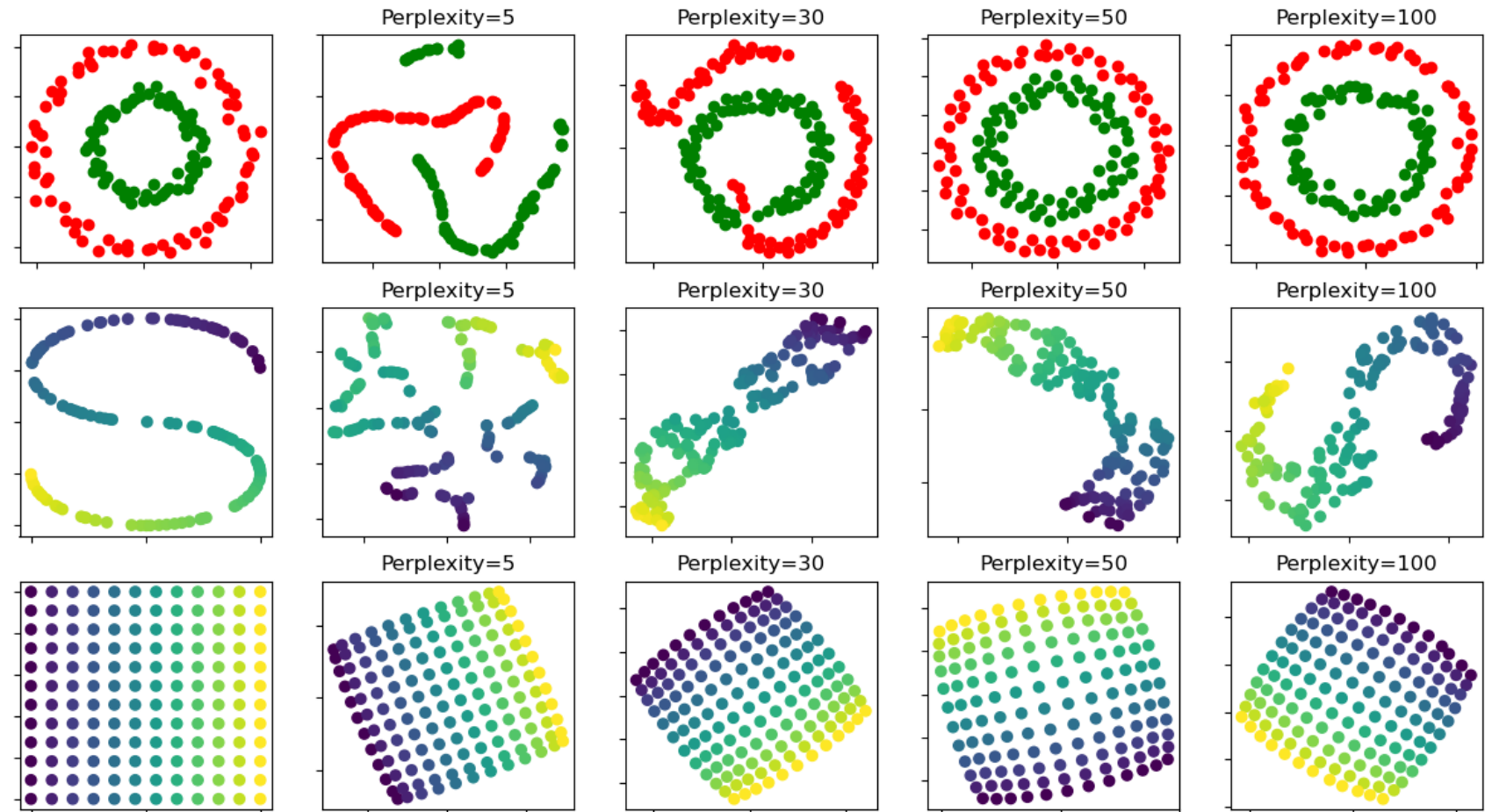


T-SNE visualisation



t-SNE perplexity hyperparameter

- The perplexity hyperparameter is a tuneable parameter that indicates how to **balance** attention between **local** and **global** aspects of the data.
- Unfortunately, the hyperparameter is, in a sense, a **guess** about the number of close neighbours each point has.
- Recommended values are between 5 and 50, being **30** the most commonly used.

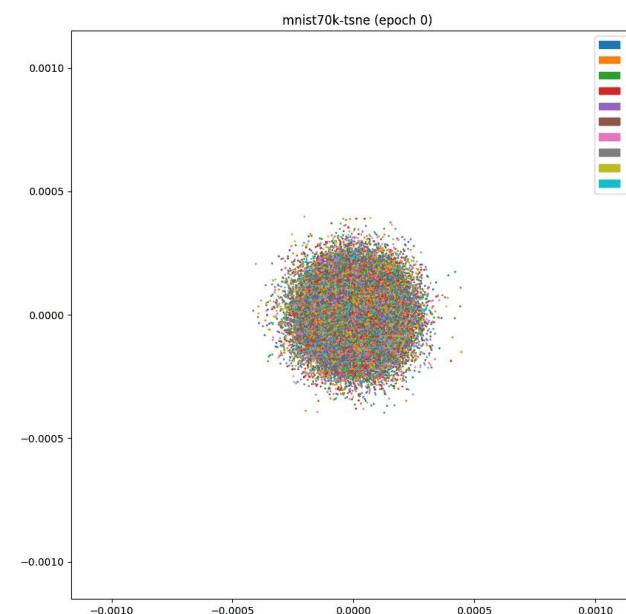
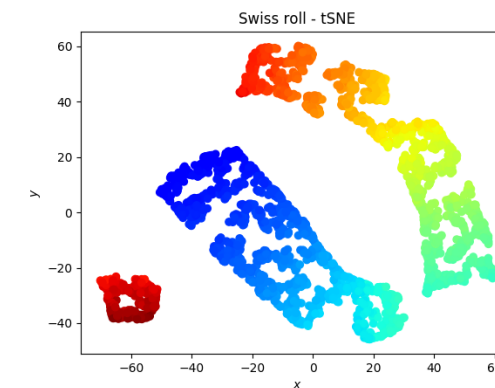
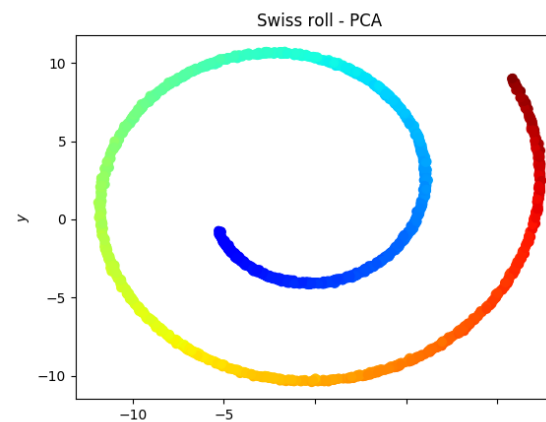
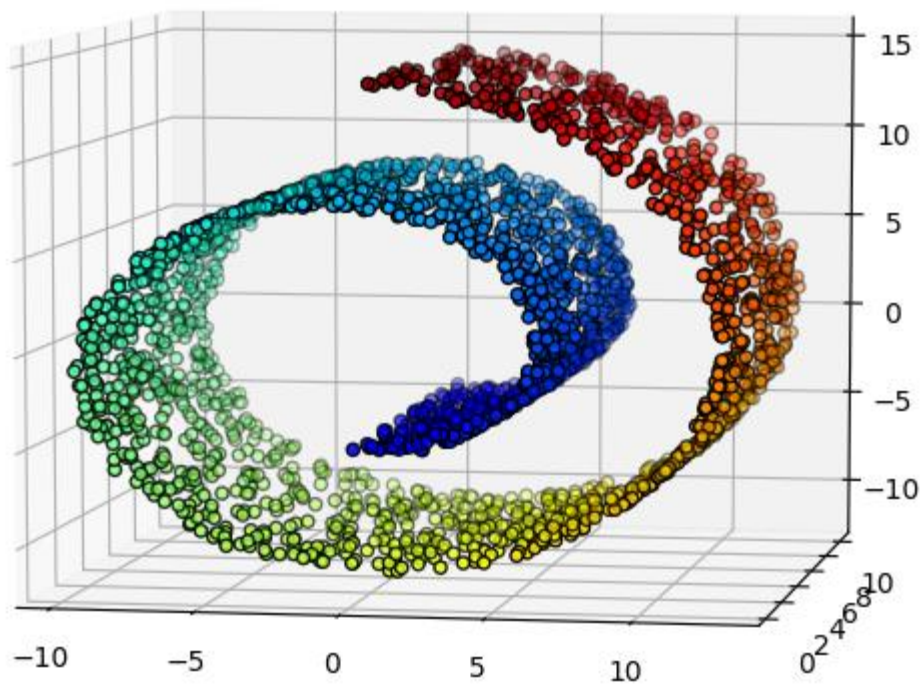


Issues with t-SNE

- t-SNE, as many other dimensionality reduction algorithms, suffers from the curse of dimensionality
 - i.e. Pairs of points will tend to have similar distance in higher dimensions.
- Depending on the data, they might not be sampled uniformly from the manifold.
 - This makes harder to embed high dimensional data into a lower dimensional space.
- t-SNE doesn't scale well.
 - For highly dimensional data ($D \gg 30$), it is recommended to perform dimensionality reduction in two stages: PCA first (to reduce dimensions to ~ 30), then, t-SNE.
- t-SNE doesn't preserve global data structure.
 - Only within cluster distances are meaningful while between cluster similarities are not guaranteed, therefore it is widely acknowledged that clustering on t-SNE is not a very good idea.
- t-SNE can practically only embed into 2 or 3 dimensions.
 - i.e. only for visualisation purposes, so it is hard to use t-SNE as a general dimension reduction technique in order to produce e.g. 10 or 50 components.

PCA vs t-SNE

Swiss roll



Uniform Manifold Approximation and Projection (UMAP)

- UMAP is an algorithm for dimension reduction based on manifold learning techniques and ideas from topological data analysis.
- It works like t-SNE. However, it is **not limited** to Euclidean distances.

$$p_{j|i} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right) \quad \text{where } \rho_i \text{ is a Riemannian structure}$$

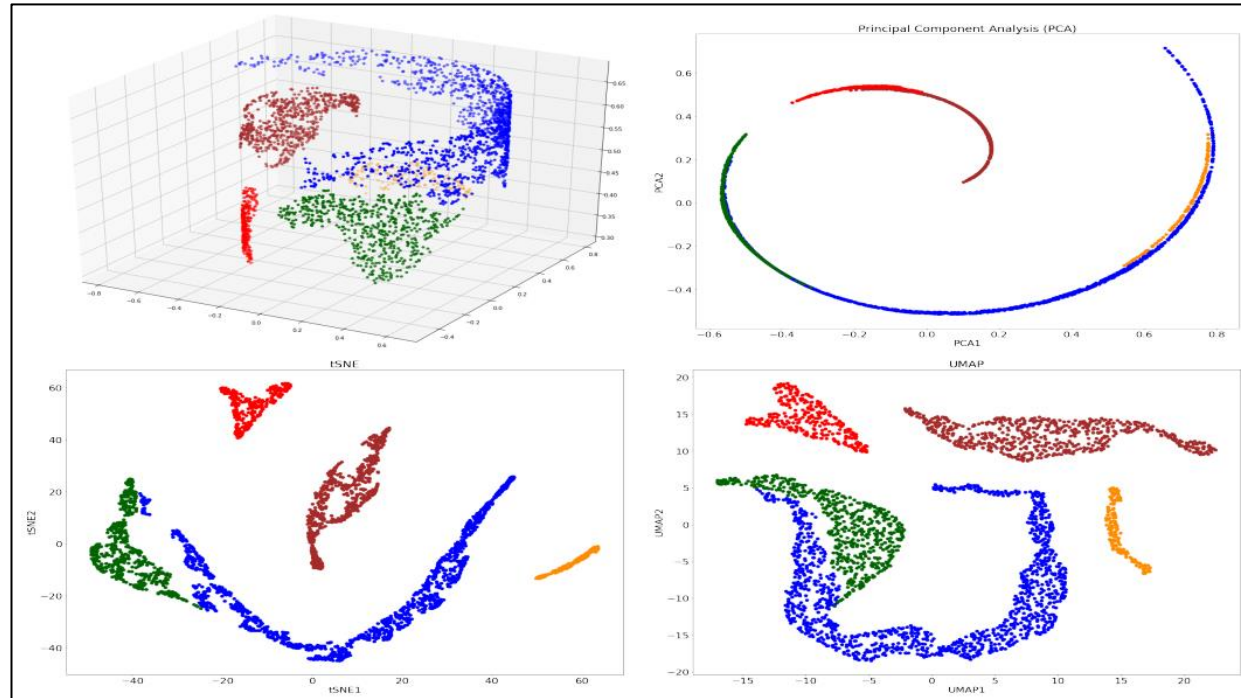
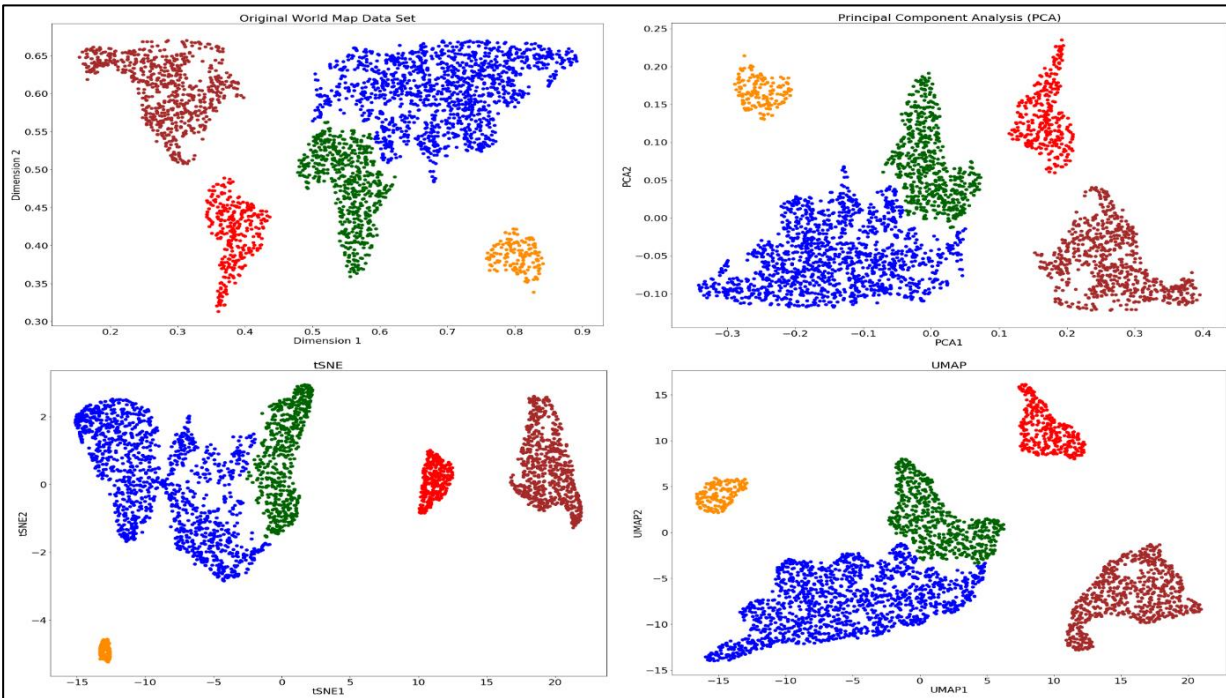
- The parameter ρ_i represents the distance from each i -th data point to its first nearest neighbour, which ensures local connectivity.
- UMAP uses a family of curves to define $q_{i|j}$, providing more flexibility to the embedding.

$$q_{j|i} = \left(1 + a\|y_i - y_j\|^{2b}\right)^{-1}$$

- UMAP uses the number of nearest neighbours instead of perplexity.

Global structure preservation

- PCA can preserve the global structure, but it is limited by its linear nature.
- t-SNE captures well local structure in non-linear data but not so well their global structure.
- UMAP captures better the global structure.



- <https://sites.duke.edu/dimensionreduction/>
- <https://pair-code.github.io/understanding-umap/>
- <https://umap-learn.readthedocs.io/en/latest/>
- <https://projector.tensorflow.org/>